

01

Welcome

02

Key Commands

03

Setting Up Claude

04

MCP Integrations

05

Skills

06

Custom Skills

07

Sub-agents

08

Hooks

09

Agent Teams

10

Browser Automation

11

Q&A

INTRO SESSION · 2026

Claude Code

AI-powered coding assistant that reads your codebase, edits files, runs commands, and integrates with your development tools.

- Terminal CLI
- VS Code
- JetBrains
- Desktop
- Web
- Chrome

```
$ claude
Scanning project...
Found CLAUDE.md
>
```

- ▲ Build features in plain language
- ▲ Write tests, fix lint, resolve conflicts
- ▲ Git commits and pull requests
- ▲ Connect tools via MCP
- ▲ Schedule recurring automation

01
Welcome

02
Key Commands

03
Setting Up Claude

04
MCP Integrations

05
Skills

06
Custom Skills

07
Sub-agents

08
Hooks

09
Agent Teams

10
Browser Automation

11
Q&A

ESSENTIALS

Useful Commands

A quick reference to get you started.

`/init`

Bootstrap CLAUDE.md — first thing in any new project

`/sandbox`

Toggle sandbox mode

`/config`

View and edit Claude Code settings and permissions

`/code-review`

Deep code review of a pull request

`/compact`

Compress context when conversation gets long

`/btw`

Quick side-task without losing current context

`/resume`

Resume a previous conversation where you left off

`/stats`

View session statistics and token usage

`! command`

Run shell inline (e.g. `! make test`)

`/insights`

Analyze sessions, interaction patterns, and friction points

Also handy: `Tab` autocomplete · `#` add files to context · `Shift+Tab` plan/act modes · `/help` all commands · [Full docs](#) →

- 01
Welcome
- 02
Key Commands
- 03
Setting Up Claude
- 04
MCP Integrations
- 05
Skills
- 06
Custom Skills
- 07
Sub-agents
- 08
Hooks
- 09
Agent Teams
- 10
Browser Automation
- 11
Q&A

SETUP

Setting Up Claude

Project instructions, rules, and settings.



CLAUDE.md

Team-shared project instructions. Coding standards, build commands, architecture. Loaded every session. Use `@path` to import other files.



settings.json

Configure permissions, environment variables, and tool behavior.



.claude/rules/

Modular rules per topic. Scoped to specific file paths — only loaded when Claude works with matching files, reducing noise and saving context.

```
# CLAUDE.md
## Tech Stack
Go 1.22, React 18, PostgreSQL
## Commands
make test — run all tests
make lint — check code style
See @docs/architecture.md for details

# .claude/rules/api-design.md
---
paths:
  - "src/api/**/*.ts"
---

All endpoints must include validation
Use standard error response format
```

Managed Policy
org-wide



Project .claude/
team-shared



User ~/.claude/
personal

Tip: Run `/init` to auto-generate your CLAUDE.md. · [Full docs →](#)

01

Welcome

02

Key Commands

03

Setting Up Claude

04

MCP Integrations

05

Skills

06

Custom Skills

07

Sub-agents

08

Hooks

09

Agent Teams

10

Browser Automation

11

Q&A

INTEGRATIONS

MCP – Connect Your Tools

Model Context Protocol lets Claude work with external services directly.



Figma

Read designs, inspect components, extract styles. Build UI directly from Figma mocks without manual spec translation.



Shortcut

Read stories, create tasks, update status. Link code changes to tickets and automate project tracking.



Slack

Read channels, search messages, send updates. Stay connected with your team without leaving the terminal.

```
# Add an MCP server
$ claude mcp add figma \
  --transport http \
  https://figma.com/mcp

$ claude mcp add shortcut \
  --transport http \
  https://shortcut.com/mcp

# Or configure in .mcp.json
{
  "mcpServers": {
    "slack": {
      "type": "http",
      "url": "https://slack.com/mcp"
    }
  }
}
```

Run `/mcp` to manage servers. 50+ integrations available. · [Full docs](#) →

- 01 Welcome
- 02 Key Commands
- 03 Setting Up Claude
- 04 MCP Integrations
- 05 Skills
- 06 Custom Skills
- 07 Sub-agents
- 08 Hooks
- 09 Agent Teams
- 10 Browser Automation
- 11 Q&A

FEATURE

Skills — Plug In & Go

Reusable prompt templates that extend Claude's capabilities. Invoke with /name or let Claude load automatically.

BUNDLED SKILLS

- ▲ **/batch** — Parallel changes across codebase via worktrees
- ▲ **/simplify** — Review changed files for reuse and quality
- ▲ **/debug** — Enable debug logging and troubleshoot
- ▲ **/loop** — Run a prompt on a recurring interval
- ▲ **/claude-api** — Load API reference for your language

[Full docs →](#)



01
Welcome

02
Key Commands

03
Setting Up Claude

04
MCP Integrations

05
Skills

06
Custom Skills

07
Sub-agents

08
Hooks

09
Agent Teams

10
Browser Automation

11
Q&A

DEEP DIVE

Building Custom Skills

Teach Claude new capabilities with a single markdown file.



SKILL.md

Entry point for every skill. YAML frontmatter controls behavior, markdown body has the instructions Claude follows.



Where They Live

Project: `.claude/skills/` (shared via git). Personal: `~/.claude/skills/` (all your projects).



Key Options

`disable-model-invocation` for manual-only. `context: fork` for subagent. `paths` to scope to files. `allowed-tools` to restrict access.

```
# .claude/skills/deploy/SKILL.md
---
name: deploy
description: Deploy to staging
disable-model-invocation: true
allowed-tools: Bash(npm *)
---

Deploy $ARGUMENTS to staging:

1. Run the test suite
2. Build the application
3. Push to deployment target
4. Verify health checks pass

# Dynamic context injection
# Shell runs before prompt
Branch: !`git branch --show`
Commit: !`git log -1 --oneline`
```

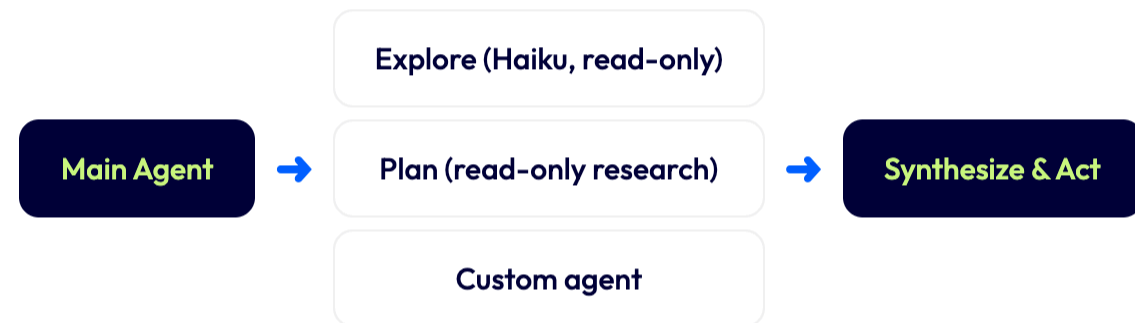
Can include supporting files: templates, scripts. [Full docs](#) →

- 01 Welcome
- 02 Key Commands
- 03 Setting Up Claude
- 04 MCP Integrations
- 05 Skills
- 06 Custom Skills
- 07 Sub-agents
- 08 Hooks
- 09 Agent Teams
- 10 Browser Automation
- 11 Q&A

FEATURE

Sub-agents — Parallel Brainpower

Each runs in its own context with custom tools, model, and permissions.



Built-in Agents

Explore (Haiku, fast search), Plan (research for planning), General-purpose (all tools, full capability).



Custom Agents

Define in `.claude/agents/`. Set model, tools, permissions, hooks, memory, and MCP servers.



Why Use Them

Keep exploration out of your main context. Isolate verbose output (tests, logs). Enforce tool restrictions. Run parallel research independently.

Run `/agents` to create and manage. [Full docs →](#)

01

Welcome

02

Key Commands

03

Setting Up Claude

04

MCP Integrations

05

Skills

06

Custom Skills

07

Sub-agents

08

Hooks

09

Agent Teams

10

Browser Automation

11

Q&A

AUTOMATION

Hooks — Automate Everything

Shell commands that run automatically at key points in Claude's lifecycle.



What They Do

Deterministic actions that always happen — not relying on Claude to choose. Format code after edits, block protected files, send notifications, validate commands.



Key Events

PreToolUse (block/allow), PostToolUse (react to changes), Stop (verify completion), Notification (desktop alerts), SessionStart (inject context).



Three Types

command — run a shell script. prompt — LLM yes/no judgment. agent — subagent that reads files and runs tools to verify.

```
# settings.json — auto-format after edits
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r
          '.tool_input.file_path'
          | xargs prettier --write"
      }]
    }]
  }
}
```

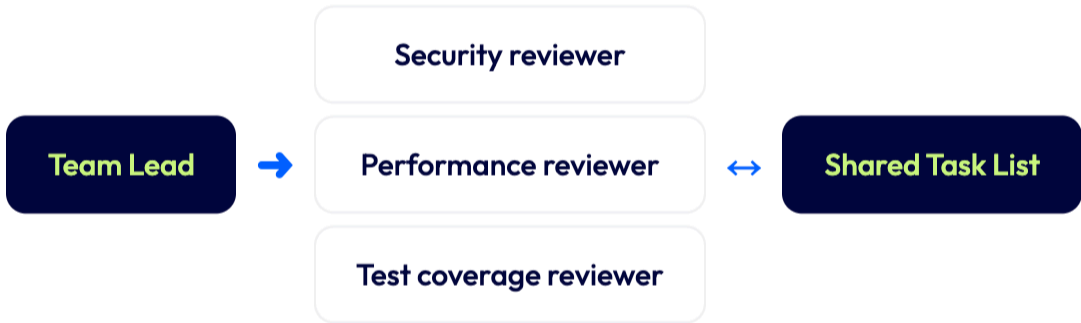
Configure in `~/.claude/settings.json` (global) or `.claude/settings.json` (project). Run `/hooks` to browse. [Full docs →](#)

- 01 Welcome
- 02 Key Commands
- 03 Setting Up Claude
- 04 MCP Integrations
- 05 Skills
- 06 Custom Skills
- 07 Sub-agents
- 08 Hooks
- 09 Agent Teams
- 10 Browser Automation
- 11 Q&A

EXPERIMENTAL

Agent Teams – Coordinated Collaboration

Multiple Claude instances working together with shared tasks and direct messaging.



How It Differs

Unlike subagents, teammates **message each other directly**, share a task list, and self-coordinate. Each is a full independent Claude session.



Best For

Parallel code review, competing hypotheses debugging, cross-layer changes (frontend + backend + tests), research from multiple angles.



Good to Know

Experimental feature. Higher token cost (each teammate has own context). Start with 3-5 teammates. Avoid same-file edits across teammates.

Enable with `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` in settings. · [Full docs](#) →

- 01 Welcome
- 02 Key Commands
- 03 Setting Up Claude
- 04 MCP Integrations
- 05 Skills
- 06 Custom Skills
- 07 Sub-agents
- 08 Hooks
- 09 Agent Teams
- 10 Browser Automation
- 11 Q&A

BETA

Browser Automation with Chrome

Build your code, then test and debug in the browser without switching contexts.



How It Works

Claude in Chrome extension connects via MCP. Opens new tabs, shares your login state. Start with `claude --chrome` or `@browser` in VS Code.



What You Can Do

Test web apps, read console errors, fill forms, extract data, draft in Google Docs, automate multi-site workflows, record GIFs.



Authenticated Access

Works with any site you're logged into — Gmail, Notion, Jira, your CRM. No API keys or OAuth setup needed.

```
# Test local app
> Open localhost:3000 and test
  the login flow
```

```
| Creating new tab...
| Navigating to localhost:3000
| Filling email field
| Clicking "Sign In"
| Taking screenshot...
```

Login successful.

```
# Debug with console
> Check the console for errors
  when the dashboard loads
```

```
# Record a demo
> Record a GIF of the checkout
  flow from cart to confirmation
```

Requires **Claude in Chrome** extension v1.0.36+. [Full docs](#) →

- 01 Welcome
- 02 Key Commands
- 03 Setting Up Claude
- 04 MCP Integrations
- 05 Skills
- 06 Custom Skills
- 07 Sub-agents
- 08 Hooks
- 09 Agent Teams
- 10 Browser Automation

11 Q&A



Any questions?

`$ claude "explain what you just heard"`

[Read the docs](#)

`npm i -g @anthropic-ai/claude-code`